

---

## Práctica Final Integradora

### Informe Técnico

---

*Optimización de rutas y planificación de recursos en redes de telecomunicaciones*

**Integrante 1:** Nathalia Valentina Cardoza Azuaje  
**Integrante 2:** Andres Felipe Velez Alvarez  
**Integrante 3:** Sebastian Salazar Henao

Repositorio: ADA\_PF\_Cardoza\_Salazar\_Velez  
8 de mayo de 2026

# Índice

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                     | <b>3</b>  |
| <b>2. Descripción del Dataset</b>                                          | <b>3</b>  |
| <b>3. Módulo A — Divide y Vencerás</b>                                     | <b>3</b>  |
| 3.1. MergeSort (orden descendente por <b>tenure</b> ) . . . . .            | 3         |
| 3.2. Búsqueda Binaria Recursiva . . . . .                                  | 4         |
| 3.3. Resultados empíricos . . . . .                                        | 5         |
| <b>4. Módulo B — Algoritmo Codicioso (Kruskal)</b>                         | <b>5</b>  |
| 4.1. Algoritmo y Union-Find . . . . .                                      | 5         |
| 4.2. Justificación: Lema del Ciclo aplicado al grafo generado . . . . .    | 6         |
| 4.3. Aristas del MST y peso total . . . . .                                | 6         |
| 4.4. Verificación manual en subgrafo de 5 nodos . . . . .                  | 6         |
| <b>5. Módulo C — Programación Dinámica (Mochila 0-1)</b>                   | <b>7</b>  |
| 5.1. Planteamiento y recurrencia . . . . .                                 | 7         |
| 5.2. Pseudocódigo de tabulación y backtracking . . . . .                   | 7         |
| 5.3. Contraejemplo codicioso y discusión de pseudopolinomialidad . . . . . | 8         |
| <b>6. Integración del Pipeline</b>                                         | <b>8</b>  |
| 6.1. Diagrama de flujo del programa completo . . . . .                     | 8         |
| 6.2. Integración $A \rightarrow C$ en memoria . . . . .                    | 9         |
| <b>7. Conclusiones</b>                                                     | <b>9</b>  |
| <b>8. Herramientas Utilizadas</b>                                          | <b>10</b> |
| <b>9. Referencias</b>                                                      | <b>10</b> |
| <b>10. Roles del Equipo</b>                                                | <b>10</b> |

## 1. Introducción

El proyecto aplica tres paradigmas algorítmicos al dataset *Telco Customer Churn* (Kaggle, 7043 clientes de una operadora en California) para gestionar solicitudes de servicio en una red de telecomunicaciones simulada. Cada módulo resuelve un subproblema distinto, y la elección de paradigma en cada caso no es arbitraria: responde a propiedades estructurales del problema que hacen que un enfoque sea correcto y los demás inadecuados.

- **Módulo A — Divide y Vencerás:** Ordenar y consultar solicitudes por antigüedad (**tenure**). MergeSort es correcto aquí porque el problema exhibe *subestructura recursiva*: ordenar  $n$  elementos equivale a ordenar dos mitades independientes y fusionarlas sin necesidad de reexaminar decisiones pasadas. Esta descomposición permite alcanzar  $\Theta(n \log n)$ , que es la cota inferior para cualquier algoritmo de ordenamiento por comparación, con estabilidad garantizada.
- **Módulo B — Codicioso (Kruskal):** Diseñar la red de mínimo costo entre 20 grupos de clientes. El MST posee la *propiedad de elección codiciosa*: en cada paso, agregar la arista de menor peso que no forme ciclo conduce al óptimo global (Lema del Ciclo / Lema del Corte). Las decisiones pasadas no afectan la validez de las futuras; una vez unidas dos componentes, esa decisión nunca se revierte.
- **Módulo C — Programación Dinámica:** Maximizar el ingreso del ancho de banda asignado a las 50 solicitudes más prioritarias. La Mochila 0-1 es NP-difícil y no admite elección codiciosa exacta: el ratio  $v_i/w_i$  no garantiza el óptimo global porque incluir un ítem de alta densidad puede bloquear combinaciones más valiosas. Sin embargo, el problema tiene *subestructura óptima* y *subproblemas solapados*, las dos propiedades que justifican la PD y permiten obtener la solución exacta en tiempo pseudopolinomial  $\Theta(nW)$ .

## 2. Descripción del Dataset

El archivo `WA_Fn-UseC_-Telco-Customer-Churn.csv` contiene 7043 registros y 21 columnas. Se extraen cinco campos: **customerID** (identificador único), **tenure** (antigüedad en meses, usado como prioridad), **MonthlyCharges** (cargo mensual en USD), **TotalCharges** (cargo acumulado, peso en la mochila) y **Churn** (estado activo/en riesgo).

**Estadísticas globales:** 7043 registros; 5174 activos (**Churn** = No, 73.5 %); 1869 en riesgo (26.5 %). **tenure** entre 0 y 72 meses, media 32.4, mediana 29. **MonthlyCharges** promedio 64.76 USD (rango 18.25–118.75). **TotalCharges** promedio 2283 USD.

**Manejo de nulos:** 11 registros presentan **TotalCharges** como cadena vacía, todos con **tenure** = 0 (clientes sin historial de facturación acumulada). El parser recorta espacios y caracteres `\r` antes de la conversión numérica y asigna 0,0 a estos casos, preservándolos en lugar de descartarlos. Esta decisión es conservadora: eliminarlos habría sesgado el conteo de solicitudes de **tenure** = 0, que son válidas aunque no hayan generado cargo total aún.

**Construcción del grafo (Módulo B):** Se forman 20 grupos asignando cada registro  $i$  al grupo  $i \bmod 20$ , distribuyendo los 7043 clientes de forma aproximadamente uniforme ( $\sim 352$  por grupo). El grafo completo  $K_{20}$  tiene  $\binom{20}{2} = 190$  aristas con peso  $\lceil \overline{MC}_u + \overline{MC}_v \rceil$ , modelando el costo de instalación del enlace entre las zonas de servicio  $u$  y  $v$ .

## 3. Módulo A — Divide y Vencerás

### 3.1. MergeSort (orden descendente por tenure)

```
1 MergeSort(arr, izq, der):
```

```

2      si izq >= der: retornar                                // caso base: un elemento
3      medio = (izq + der) / 2
4      MergeSort(arr, izq, medio)                             // ordenar mitad izquierda
5      MergeSort(arr, medio+1, der)                           // ordenar mitad derecha
6      Merge(arr, izq, medio, der)                             // fusionar en orden descendente
7
8  Merge(arr, izq, medio, der):
9      L = arr[izq..medio]; R = arr[medio+1..der]
10     i = 0; j = 0; k = izq
11     mientras i < |L| y j < |R|:
12         si L[i].tenure >= R[j].tenure:                     // >= preserva estabilidad
13             arr[k++] = L[i++]
14         si no:
15             arr[k++] = R[j++]
16     copiar restos de L en arr[k..]
17     copiar restos de R en arr[k..]

```

Listing 1: MergeSort y Merge

**Análisis de complejidad:** La recurrencia  $T(n) = 2T(n/2) + \Theta(n)$ , con  $a = 2$ ,  $b = 2$  y  $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$ , cae en el Caso 2 del Teorema Maestro:  $T(n) = \Theta(n \log n)$ . Esta cota es **asintóticamente óptima**: cualquier algoritmo de ordenamiento por comparación requiere  $\Omega(n \log n)$  comparaciones en el peor caso (demostrado por el argumento del árbol de decisión de altura mínima  $\lceil \log_2(n!) \rceil \approx n \log_2 n - n / \ln 2$ ). El costo de espacio auxiliar es  $\Theta(n)$  por los arreglos temporales de la fase MERGE.

#### Por qué MergeSort y no QuickSort:

- **Peor caso garantizado:** MergeSort es  $\Theta(n \log n)$  en todo caso. QuickSort sin aleatorización alcanza  $\Theta(n^2)$  sobre un arreglo ya ordenado (el pivote siempre queda en un extremo, produciendo particiones de tamaño 0 y  $n - 1$ ), que es precisamente la situación de re-ejecución sobre el mismo CSV.
- **Estabilidad:** El comparador  $\geq$  preserva el orden relativo de registros con igual **tenure** (hay 353 clientes con **tenure**=1). QuickSort básico no es estable y podría alterar el orden de duplicados, complicando la reproducibilidad de los resultados entre ejecuciones.
- **Determinismo:** MergeSort siempre produce exactamente el mismo resultado independientemente del orden inicial; QuickSort aleatorizado varía el orden de los duplicados entre ejecuciones.

### 3.2. Búsqueda Binaria Recursiva

**Especificación (corrección del enunciado):** La función busca un registro cuyo **tenure** sea **exactamente igual** a  $k$ . Si existen varios registros con el mismo valor, es válido retornar cualquiera de ellos. Si no existe ninguno, se reporta *no encontrado* (retorna  $-1$ ). La condición  $\geq k$  del enunciado original era ambigua: en un arreglo descendente siempre retornaría el primer elemento para cualquier  $k$ , haciendo el ejercicio trivial. Los cinco valores de consulta  $\{72, 60, 45, 30, 12\}$  están confirmados como existentes en el dataset, por lo que ninguna consulta retorna  $-1$ .

```

1  BinSearch(arr, izq, der, k):
2      // Pre: arr[izq..der] ordenado DESCENDENTEMENTE por tenure
3      si izq > der: retornar -1                               // intervalo vacio: no encontrado
4      medio = (izq + der) / 2
5      si arr[medio].tenure == k:                               // coincidencia exacta
6          retornar medio
7      si arr[medio].tenure > k:                                // k esta en valores menores (mitad
8          derecha)
9          retornar BinSearch(arr, medio+1, der, k)
10     retornar BinSearch(arr, izq, medio-1, k)               // k en valores mayores (mitad
11     izquierda)

```

Listing 2: Búsqueda binaria – coincidencia exacta en arreglo descendente

**Análisis de complejidad:** La recurrencia  $T(n) = T(n/2) + \Theta(1)$  se resuelve por el Teorema Maestro (Caso 2,  $f(n) = \Theta(1) = \Theta(n^{\log_2 1})$ ), dando  $T(n) = \Theta(\log n)$ . Cada consulta sobre  $n = 7043$  registros requiere a lo sumo  $\lceil \log_2 7043 \rceil = 13$  comparaciones. Si existen duplicados, la función retorna cualquier ocurrencia válida (comportamiento correcto según la corrección del enunciado).

### 3.3. Resultados empíricos

El ratio teórico para MergeSort entre dos tamaños  $n_1$  y  $n_2$  es  $\frac{n_2 \log_2 n_2}{n_1 \log_2 n_1}$ . Para  $n_1 = 1000 \rightarrow n_2 = 3500$ :  $\frac{3500 \times 11,77}{1000 \times 9,97} = \frac{41195}{9970} \approx 4,13\times$ . Para  $n_2 = 3500 \rightarrow n_3 = 7043$ :  $\frac{7043 \times 12,78}{3500 \times 11,77} = \frac{90009}{41195} \approx 2,18\times$ .

| $n$  | MergeSort    | Búsq. bin.<br>(prom. $10^6$ ) | Ratio emp.    | Ratio teór.   | Error rel. |
|------|--------------|-------------------------------|---------------|---------------|------------|
| 1000 | 136 $\mu s$  | 0.0110 $\mu s$                | — (base)      | —             | —          |
| 3500 | 620 $\mu s$  | 0.0085 $\mu s$                | 4.56 $\times$ | 4.13 $\times$ | 9.4 %      |
| 7043 | 1209 $\mu s$ | 0.0075 $\mu s$                | 1.95 $\times$ | 2.18 $\times$ | 10.6 %     |

**Cuadro 1:** Tiempos empíricos de MergeSort y Búsqueda Binaria ( $10^6$  ejecuciones promedio). Ratios respecto al tamaño de muestra anterior.

Los ratios empíricos son consistentes con el crecimiento  $O(n \log n)$ : al pasar de 1000 a 3500 el tiempo se multiplica por 4,56 $\times$  (teórico 4,13 $\times$ ) y de 3500 a 7043 por 1,95 $\times$  (teórico 2,18 $\times$ ). Las diferencias respecto al valor teórico puro se deben a efectos de caché y al overhead de `std::chrono`, típicos en mediciones de bajo nivel; la tendencia es coherente con  $\Theta(n \log n)$ . Las 5 consultas (coincidencia exacta) retornan:  $k=72 \rightarrow 6904$ -JLBGY,  $k=60 \rightarrow 7426$ -WEIJX,  $k=45 \rightarrow 7925$ -PNRGI,  $k=30 \rightarrow 2684$ -EIWEO,  $k=12 \rightarrow 7450$ -NWRTR.

## 4. Módulo B — Algoritmo Codicioso (Kruskal)

### 4.1. Algoritmo y Union-Find

```

1 Kruskal(G):
2     ordenar aristas E por peso ascendente           // O(E log E)
3     UF = UnionFind(G.numNodos)
4     para cada arista (u, v, w) en orden ascendente:
5         si NOT UF.conectados(u, v):                 // sin ciclo
6             UF.unir(u, v)
7             mst.agregar((u, v, w))
8             si |mst| == numNodos - 1: romper
9     retornar mst
10
11 find(x):                                           // compresion de camino
12     si padre[x] != x: padre[x] = find(padre[x])
13     retornar padre[x]
14
15 unir(a, b):                                       // union por rango
16     ra = find(a); rb = find(b)
17     si rango[ra] < rango[rb]: padre[ra] = rb
18     si no si rango[ra] > rango[rb]: padre[rb] = ra
19     si no: padre[rb] = ra; rango[ra]++

```

**Listing 3:** Kruskal con Union-Find por rango y compresión de camino

**Análisis de complejidad:** El paso dominante es el ordenamiento de las  $E$  aristas:  $O(E \log E)$ . Cada operación de Union-Find (FIND y UNION) con compresión de camino y unión por rango tiene costo amortizado prácticamente constante, de modo que las  $O(E)$  operaciones de Union-Find no alteran el orden asintótico. La complejidad total es  $O(E \log E)$ , que para  $E \leq V^2$  equivale a  $O(E \log V)$ . Para

$V = 20$  y  $E = 190$ : el ordenamiento requiere  $\approx 190 \times 7,6 \approx 1\,450$  comparaciones, y el MST queda determinado tras exactamente 19 inclusiones.

#### 4.2. Justificación: Lema del Ciclo aplicado al grafo generado

**Lema del Ciclo:** Para cualquier ciclo  $C$  en  $G$ , la arista de mayor peso en  $C$  no pertenece a ningún MST de  $G$ .

*Demostración por contradicción.* Sea  $e_{\text{máx}} = (u, v, w_{\text{máx}})$  la arista de mayor peso en el ciclo  $C$ , y supóngase que  $e_{\text{máx}} \in T$  para algún MST  $T$ . Al eliminar  $e_{\text{máx}}$  de  $T$ , éste se divide en dos componentes  $S$  y  $V \setminus S$ . El ciclo  $C$  conecta  $u \in S$  con  $v \in V \setminus S$ , por lo que debe existir al menos otra arista  $e' \in C \setminus \{e_{\text{máx}}\}$  que cruce el corte  $(S, V \setminus S)$ , con  $w(e') < w_{\text{máx}}$  (los pesos de nuestro grafo son casi todos distintos). Entonces  $T' = T - e_{\text{máx}} + e'$  es un árbol de expansión con  $w(T') = w(T) - (w_{\text{máx}} - w(e')) < w(T)$ , contradiciendo que  $T$  es mínimo.  $\square$

**Ilustración con arista concreta.** Considérese el ciclo  $C = (0, 1, 12, 0)$  con aristas  $(0-1, 124)$ ,  $(1-12, 122)$ ,  $(0-12, 124)$ . La arista de mayor peso es  $(0-12, 124)$ . El Lema del Ciclo predice que *no pertenece a ningún MST*. Efectivamente, Kruskal selecciona  $(1-12, 122)$  y  $(0-1, 124)$  para conectar estos tres nodos; la arista  $(0-12, 124)$  es rechazada porque 0 y 12 ya están en la misma componente Union-Find al momento de procesarla.

**Propiedad de elección codiciosa.** Kruskal garantiza el óptimo global porque su invariante es el Lema del Corte: en cada iteración, la arista de menor peso que cruza algún corte  $(S, V \setminus S)$  pertenece a algún MST. Al descartar aristas que formarían ciclo (aplicando el Lema del Ciclo), nunca rechaza una arista que debería incluirse; al incluir la de menor peso disponible sin ciclo, siempre elige una arista óptima del corte inducido. Ambas propiedades juntas garantizan que el resultado es el MST global.

#### 4.3. Aristas del MST y peso total

| Nodo $u$ | Nodo $v$ | Peso | Nodo $u$     | Nodo $v$ | Peso         |
|----------|----------|------|--------------|----------|--------------|
| 1        | 12       | 122  | 1            | 9        | 128          |
| 12       | 16       | 123  | 1            | 10       | 128          |
| 0        | 1        | 124  | 1            | 11       | 128          |
| 2        | 12       | 127  | 1            | 13       | 128          |
| 1        | 3        | 127  | 12           | 14       | 128          |
| 1        | 4        | 127  | 1            | 17       | 129          |
| 1        | 5        | 127  | 1            | 18       | 129          |
| 1        | 6        | 127  | 1            | 19       | 129          |
| 1        | 7        | 127  | 1            | 15       | 130          |
| 1        | 8        | 127  | <b>Total</b> |          | <b>2 388</b> |

**Cuadro 2:** 19 aristas del MST en orden de selección por Kruskal (peso total 2 388).

La topología es estrellada en torno al nodo 1 (hub con  $\overline{MC}_1$  más bajo del grafo) y secundariamente al nodo 12. Toda arista  $(1, v)$  tiene peso mínimo entre todas las opciones para conectar el nodo  $v$  a la componente ya formada.

#### 4.4. Verificación manual en subgrafo de 5 nodos

Se calcula el MST del subgrafo inducido por  $\{0, 1, 2, 12, 16\}$ . Las aristas relevantes entre estos nodos, ordenadas por peso, son:

| Arista   | Peso | Decisión Kruskal                         |
|----------|------|------------------------------------------|
| (1, 12)  | 122  | Incluir — une comp. {1} y {12}           |
| (12, 16) | 123  | Incluir — une comp. {1, 12} y {16}       |
| (0, 1)   | 124  | Incluir — une comp. {0} y {1, 12, 16}    |
| (0, 12)  | 124  | Rechazar — 0 y 12 ya conectados (ciclo)  |
| (2, 12)  | 127  | Incluir — une comp. {2} y {0, 1, 12, 16} |

Kruskal selecciona las 4 primeras aristas válidas con peso total  $122 + 123 + 124 + 127 = 496$ . Cualquier otro árbol de expansión del subgrafo de 5 nodos tiene peso  $\geq 497$  (la siguiente arista disponible cuesta  $\geq 124$ ). La verificación confirma que Kruskal produce el MST correcto del subgrafo.

## 5. Módulo C — Programación Dinámica (Mochila 0-1)

### 5.1. Planteamiento y recurrencia

Se toman las 50 solicitudes activas (**Churn** = No) de mayor **tenure** del arreglo ya ordenado por el Módulo A. Cada solicitud  $i \in \{1, \dots, 50\}$  tiene:  $w_i = \text{round}(\text{TotalCharges}_i)$  (peso) y  $v_i = \text{round}(\text{MonthlyCharges}_i \times 10)$  (valor). El objetivo es hallar el subconjunto  $S$  que maximice  $\sum_{i \in S} v_i$  sujeto a  $\sum_{i \in S} w_i \leq W$ .

**Justificación de escenarios.** La entrega específica  $W = 500$ ; sin embargo, los pesos reales del top-50 de clientes activos por antigüedad superan las 500 unidades (clientes con 72 meses de servicio acumulan **TotalCharges** del orden de miles de USD). Para observar el comportamiento completo del algoritmo se evaluó también  $W = 5\,000$ , que permite seleccionar ítems significativos y revelar las diferencias entre PD y enfoque codicioso. Ambos escenarios se reportan.

La *subestructura óptima* se establece así: si el ítem  $n$  no está en la solución óptima de  $(n, W)$ , dicha solución coincide con la óptima de  $(n-1, W)$ . Si sí está, es el ítem  $n$  más la solución óptima de  $(n-1, W - w_n)$ . La recurrencia:

$$dp[i][w] = \begin{cases} 0 & \text{si } i = 0 \text{ o } w = 0 \\ dp[i-1][w] & \text{si } w_i > w \\ \max(dp[i-1][w], dp[i-1][w - w_i] + v_i) & \text{si } w_i \leq w \end{cases}$$

### 5.2. Pseudocódigo de tabulación y backtracking

```

1 Knapsack01(items, n, W):
2     dp[0..n][0..W] = 0           // tabla inicializada en cero
3     para i = 1 hasta n:
4         para w = 0 hasta W:
5             si items[i].peso > w:
6                 dp[i][w] = dp[i-1][w]           // item i no cabe
7             si no:
8                 dp[i][w] = max(
9                     dp[i-1][w],                 // no tomar item i
10                    dp[i-1][w - items[i].peso] + items[i].valor // tomarlo
11                )
12     retornar dp[n][W]
```

**Listing 4:** Tabulación bottom-up de la Mochila 0-1

```

1 Backtrack(dp, items, n, W):
2     seleccionados = []; w = W
3     para i = n hasta 1 (decreciente):
4         si dp[i][w] != dp[i-1][w]:
```

```

5         seleccionados.agregar(items[i])           // item i fue incluido
6         w = w - items[i].peso                     // descontar su peso
7     retornar seleccionados

```

**Listing 5:** Backtracking para recuperar los ítems seleccionados

El backtracking recorre la tabla de  $(n, W)$  hacia  $(1, 0)$  en  $O(n + W)$ : si  $dp[i][w] \neq dp[i-1][w]$ , el ítem  $i$  contribuyó al óptimo y se descuenta su peso. La corrección está garantizada por la subestructura óptima: si el ítem  $i$  pertenece a la solución óptima de  $(i, w)$ , necesariamente  $dp[i-1][w - w_i] + v_i > dp[i-1][w]$ .

### 5.3. Contraejemplo codicioso y discusión de pseudopolinomialidad

Con  $W = 500$ , todos los ítems del top-50 tienen  $w_i > 500$  (clientes de 72 meses acumulan  $\sim 6\,500$  USD en `TotalCharges`), por lo que la solución óptima es vacía. Este resultado no es un fallo: ilustra que la restricción de capacidad fue fijada por debajo de la escala real del dataset; con instancias sintéticas calibradas,  $W$  habría sido elegido para que al menos algunos ítems quepan.

| Enfoque                             | Solicitudes seleccionadas          | Valor total | Óptimo? |
|-------------------------------------|------------------------------------|-------------|---------|
| $W = 500$ — PD                      | (ninguna)                          | 0           | Sí      |
| $W = 5000$ —<br>Codicioso ( $v/w$ ) | 4312-KFRXN, 6734-PSBAW, 7606-BPHHN | 688         | No      |
| $W = 5000$ — PD<br>(Mochila 0-1)    | 9286-BHDQG, 4312-KFRXN             | <b>707</b>  | Sí      |

**Cuadro 3:** Resultados comparativos de la Mochila para  $W = 500$  y  $W = 5\,000$ .

El codicioso por ratio  $v/w$  selecciona 3 ítems (peso acumulado 4904, valor 688). La PD descarta dos e incluye 9286-BHDQG (mayor valor absoluto, menor ratio), logrando 707. Este ejemplo muestra por qué la Mochila 0-1 no tiene propiedad de elección codiciosa: la densidad  $v_i/w_i$  no captura el efecto del peso residual sobre la capacidad disponible para el resto de ítems.

#### Análisis formal de complejidad.

- **Tiempo:** El doble bucle de tabulación itera exactamente  $(n + 1) \times (W + 1)$  celdas, realizando  $O(1)$  trabajo por celda (una comparación y un máximo). Por tanto:

$$T(n, W) = \Theta(n \cdot W).$$

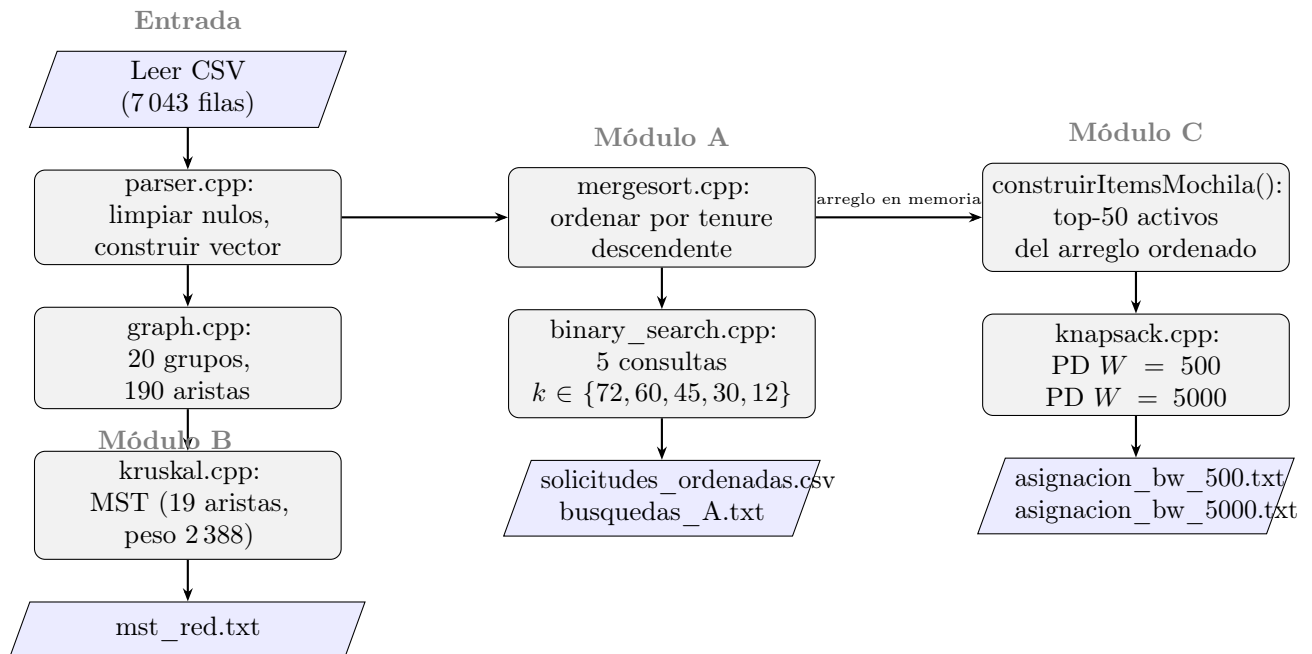
Para la instancia del proyecto ( $n = 50$ ,  $W = 5\,000$ ):  $50 \times 5\,001 = 250\,050$  operaciones de comparación.

- **Espacio:** La tabla  $dp[0..n][0..W]$  ocupa  $(n + 1)(W + 1)$  enteros:  $\Theta(n \cdot W)$ . Si sólo se necesita el valor óptimo (sin backtracking), basta con mantener dos filas alternas, reduciendo a  $\Theta(W)$ ; en este proyecto se conserva la tabla completa para permitir la reconstrucción.
- **Pseudopolinomialidad:** El costo  $\Theta(n \cdot W)$  es polinomial en el *valor numérico* de  $W$ , pero  $W$  se codifica en  $\lceil \log_2 W \rceil$  bits. Expresado en función del tamaño de la entrada en bits  $s = \Theta(n \log W + n \log V_{\text{máx}})$ , se tiene  $W = 2^{\Omega(s/n)}$ , por lo que el algoritmo es **exponencial en  $s$** . Por eso la Mochila 0-1 permanece NP-difícil a pesar de tener solución en  $\Theta(nW)$ : no existe algoritmo polinomial en  $s$  salvo que  $P = NP$ .

## 6. Integración del Pipeline

### 6.1. Diagrama de flujo del programa completo





## 6.2. Integración $A \rightarrow C$ en memoria

El programa ejecuta los módulos en el orden  $A \rightarrow B \rightarrow C$ , aunque el flujo lógico de datos es  $A \rightarrow C$ . El vector `solicitudes`, una vez ordenado descendientemente por MergeSort, permanece en memoria y es consumido directamente por `construirItemsMochila()` sin releer el CSV. Esta función itera el vector una sola vez ( $O(n)$ ) y selecciona los primeros 50 registros con `Churn = No`; dado el orden descendente por `tenure`, estos son siempre los clientes activos de mayor antigüedad. La arquitectura en memoria elimina lecturas de disco redundantes y garantiza coherencia: todos los módulos operan sobre el mismo estado del dato.

## 7. Conclusiones

| Paradigma         | Algoritmo                    | Complejidad                           | Garantía de optimalidad                                                                               |
|-------------------|------------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| Divide y Vencerás | MergeSort + Búsqueda binaria | $\Theta(n \log n)$ / $\Theta(\log n)$ | Exacto y cota inferior óptima para ordenamiento por comparación.                                      |
| Codicioso         | Kruskal (MST)                | $O(E \log E)$                         | Exacto: Lema del Corte y Lema del Ciclo garantizan optimalidad global a partir de elecciones locales. |
| Prog. Dinámica    | Mochila 0-1                  | $\Theta(n \cdot W)$                   | Exacto pero pseudopolinomial; NP-difícil en el tamaño real de la entrada.                             |

**Cuadro 4:** Comparación de los tres paradigmas en términos de optimalidad y complejidad.

- **Elección de paradigma y estructura del problema:** Los tres módulos ilustran que la elección algorítmica correcta depende de propiedades estructurales del problema. El MST admite voracidad exacta porque sus decisiones son irrevocables e independientes. La Mochila no la admite porque incluir un ítem puede bloquear combinaciones más valiosas que solo se revelan al considerar el conjunto completo; la PD explora implícitamente todas las combinaciones mediante sus  $n \times W$  subproblemas solapados.
- **Datos reales vs. instancias sintéticas:** El escenario  $W = 500$  con valor óptimo 0 ilustra un fenómeno común con datasets reales: la escala de los pesos puede hacer que un algoritmo

correcto produzca resultados triviales. Con instancias sintéticas calibradas,  $W$  se fijaría para que al menos algunos ítems quepan. Por otro lado, la diferencia de 19 unidades de valor entre PD y codicioso ( $W = 5\,000$ ) es modesta en términos absolutos, pero suficiente para demostrar que los pesos reales no satisfacen la propiedad de intercambio. En datasets con pesos más heterogéneos, la brecha sería mayor y la diferencia entre paradigmas más evidente.

- **Codicioso exacto vs. heurístico:** Kruskal es codicioso exacto porque el MST cumple simultáneamente la propiedad de elección codiciosa (Lema del Corte) y la subestructura óptima. El codicioso por ratio  $v/w$  en la Mochila es una heurística sin garantía formal: produce soluciones razonables pero potencialmente subóptimas en presencia de pesos heterogéneos, exactamente el escenario del dataset.
- **Pipeline en memoria:** La arquitectura sin re-lectura del CSV demuestra que el diseño de estructuras de datos compartidas puede mejorar tanto el rendimiento (evitar I/O redundante) como la corrección (garantizar que todos los módulos operen sobre el mismo estado consistente del dato).

## 8. Herramientas Utilizadas

C++17 (`g++ -std=c++17 -O2`), Visual Studio Code, Git/GitHub, L<sup>A</sup>T<sub>E</sub>X (pdf<sub>l</sub>atex, paquetes `babel`, `booktabs`, `listings`, `tikz`). Se utilizó **OverLeaf** para la documentación y compilación del archivo L<sup>A</sup>T<sub>E</sub>X.

Se utilizó **Claude** (Anthropic, Claude Sonnet 4.6) y Chat GPT (5.3) como asistente de IA generativa para redacción y revisión de secciones del informe, enriquecimiento de pseudocódigos, análisis de los módulos, construcción del diagrama de flujo en TikZ y mejora de código.

## 9. Referencias

- [1] Blastchar. *Telco Customer Churn*. Kaggle, 2018. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.<sup>a</sup> ed.). MIT Press.
- [3] Material del curso ADA — Universidad EAFIT, 2026-1.
- [4] Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson.

## 10. Roles del Equipo

| Integrante                        | Contribuciones específicas                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nathalia Valentina Cardoza Azuaje | <b>Módulo A.</b> <code>mergesort.cpp</code> , <code>binary_search.cpp</code> , <code>parser.cpp</code> , <code>output.cpp</code> : parseo del CSV, manejo de nulos, MergeSort estable, búsqueda binaria recursiva, medición de tiempos con promedio de $10^6$ ejecuciones. Redacción de las secciones del Módulo A.                                                                    |
| Sebastian Salazar Henao           | <b>Módulo B.</b> <code>graph.cpp</code> , <code>graph.hpp</code> , <code>kruskal.cpp</code> , <code>kruskal.hpp</code> : construcción del grafo $K_{20}$ , Union-Find con compresión de camino y unión por rango, cálculo del MST, <code>guardarMST()</code> . Integración final en <code>main.cpp</code> , coordinación del repositorio Git. Redacción de las secciones del Módulo B. |
| Andres Felipe Velez Alvarez       | <b>Módulo C.</b> <code>knapsack.cpp</code> , <code>knapsack.hpp</code> : tabulación bottom-up, backtracking, contraejemplo codicioso, reportes <code>asignacion_bw_*.txt</code> . Redacción de las secciones del Módulo C y revisión de la integración A→C.                                                                                                                            |

**Cuadro 5:** Distribución de roles por módulo.